

Relazione di Laboratorio

Esplorazione del Traffico DNS con Wireshark

Autore	Alessandro Porfirio
Corso	Cyber Security & Ethical Hacking – Epicode
Ambiente	Kali Linux, interfaccia di rete eth0, Wireshark
Data	17 Giugno 2026

1. Obiettivo e Contesto

In questa esercitazione ho utilizzato Wireshark per catturare ed esaminare il traffico DNS generato dalla mia macchina Kali Linux durante una risoluzione di nome tramite il comando nslookup. L'obiettivo è stato comprendere nel dettaglio la struttura di una query DNS e della relativa risposta, osservando indirizzi MAC e IP coinvolti, porte utilizzate, flag di controllo del protocollo DNS e la catena di record CNAME/A restituita dal resolver.

Il laboratorio è stato suddiviso in tre parti: cattura del traffico DNS, analisi del pacchetto di query e, infine, analisi del pacchetto di risposta.

2. Parte 1 – Catturare il Traffico DNS

Wireshark era già installato sulla mia macchina Kali, pertanto non è stato necessario il passaggio di installazione previsto dalla traccia per Windows/macOS. Ho avviato Wireshark selezionando l'interfaccia di rete attiva (eth0). Poiché la traccia specifica che su Kali la pulizia della cache DNS non è un passaggio necessario, sono passato direttamente all'esecuzione della query.

Ho avviato la cattura, aperto un terminale ed eseguito nslookup in modalità interattiva, interrogando il nome di dominio www.cisco.com:

```
Session Actions Edit View Help
(kali@kali)-[~]
$ nslookup
> www.cisco.com
Server:      192.168.60.1
Address:     192.168.60.1#53

Non-authoritative answer:
www.cisco.com canonical name = www.cisco.com.akadns.net.
www.cisco.com.akadns.net canonical name = wwwds.cisco.com.edgekey.net.
wwwds.cisco.com.edgekey.net canonical name = wwwds.cisco.com.edgekey.net.globalredir.akadns.net.
wwwds.cisco.com.edgekey.net.globalredir.akadns.net canonical name = e2867.dsca.akamaiedge.net.
Name:      e2867.dsca.akamaiedge.net
Address:   184.25.52.119
Name:      e2867.dsca.akamaiedge.net
Address:   2a02:26f0:2d:a92::b33
Name:      e2867.dsca.akamaiedge.net
Address:   2a02:26f0:2d:a81::b33
>
```

Fig. 1 – Esecuzione di nslookup per www.cisco.com dal terminale di Kali.

Terminata la query (comando exit), ho interrotto la cattura in Wireshark e applicato il filtro `udp.port == 53` per isolare unicamente il traffico DNS dalla cattura completa.

No.	Time	Source	Destination	Protocol	Length	Info
12	32.852136438	192.168.60.3	192.168.60.1	DNS	73	Standard query 0xe66b A www.cisco.com
13	32.903492784	192.168.60.1	192.168.60.3	DNS	255	Standard query response 0xe66b A www.cisco.com CNAME www.cisco.com.akadns.net CNAME wwwds.cisco.com.akadns.net CNAME wwwds.cisco.com.akadns.net
14	32.904304809	192.168.60.3	192.168.60.1	DNS	85	Standard query 0xfbe5 AAAA e2867.dsca.akamaiedge.net
15	32.921962762	192.168.60.1	192.168.60.3	DNS	141	Standard query response 0xfbe5 AAAA e2867.dsca.akamaiedge.net AAAA 2a02:26f0:2d:a92::b33 AAAA 2a02:26f0:2d:a81::b33

Fig. 2 – Traffico DNS isolato con il filtro udp.port == 53: query e risposta per www.cisco.com (A e AAAA).

Il filtro ha isolato quattro pacchetti: una query e la relativa risposta per il record di tipo A, seguite da una seconda coppia query/risposta per il record AAAA (IPv6) dello stesso nome, generata automaticamente da nslookup.

3. Parte 2 – Esplorare il Traffico delle Query DNS

Ho selezionato il pacchetto numero 12, contenente "Standard query 0xe66b A www.cisco.com" nella colonna Info, ed espanso le sezioni Ethernet II, Internet Protocol Version 4 e User Datagram Protocol nel riquadro dei dettagli del pacchetto.

<pre> - Frame 12: Packet, 73 bytes on wire (584 bits), 73 bytes captured (584 bits) on interface eth0, id 0 Section number: 1 Interface id: 0 (eth0) Encapsulation type: Ethernet (1) Arrival Time: Jun 17, 2020 10:44:07.388582508 EDT UTC Arrival Time: Jun 17, 2020 14:44:07.388582508 UTC Epoch Arrival Time: 1781707447.388582508 [Time shift for this packet: 0.000000000 seconds] [Time delta from previous captured frame: 888.839534 milliseconds] [Time since reference or first frame: 32.852136438 seconds] Frame Number: 12 Frame Length: 73 bytes (584 bits) Capture Length: 73 bytes (584 bits) [Frame is marked: False] [Frame is ignored: False] [Protocols in frame: ethII:ethertype:ip:udp:dns] Character encoding: ASCII (0) [Coloring Rule Name: udp] [Coloring Rule String: udp] - Ethernet II, Src: Se:fd:b2:0a:85:05 (Se:fd:b2:0a:85:05), Dst: 86:2f:57:14:c9:64 (86:2f:57:14:c9:64) Destination: 86:2f:57:14:c9:64 (86:2f:57:14:c9:64) Source: Se:fd:b2:0a:85:05 (Se:fd:b2:0a:85:05) Type: IPv4 (0x0800) [Stream index: 0] - Internet Protocol Version 4, Src: 192.168.60.3, Dst: 192.168.60.1 0100 = Version: 4 0101 = Header Length: 20 bytes (5) Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT) Total Length: 59 Identification: 0x60c9 (24777) 0000 = Flags: 0x0 ...0 0000 0000 0000 = Fragment Offset: 0 Time to Live: 64 Protocol: UDP (17) Header Checksum: 0x2094 [validation disabled] [Header checksum status: Unverified] Source Address: 192.168.60.3 Destination Address: 192.168.60.1 [Stream index: 2] </pre>	
---	--

Fig. 3 – Dettaglio del Frame 12: intestazioni Ethernet II e IPv4 della query DNS.

Indirizzi MAC di origine e destinazione	Origine: 5e:fd:b2:0a:85:05 — Destinazione: 86:2f:57:14:c9:64
A quali interfacce sono associati	L'indirizzo di origine corrisponde all'interfaccia eth0 della mia macchina Kali; quello di destinazione corrisponde al gateway/resolver DNS della subnet virtuale (192.168.60.1), che funge anche da router predefinito.
Indirizzi IP di origine e destinazione	Origine: 192.168.60.3 — Destinazione: 192.168.60.1
A quali interfacce sono associati	192.168.60.3 è l'IP assegnato all'interfaccia eth0 della mia Kali; 192.168.60.1 è il gateway/server DNS della rete (broadcast 192.168.60.255, subnet /24).

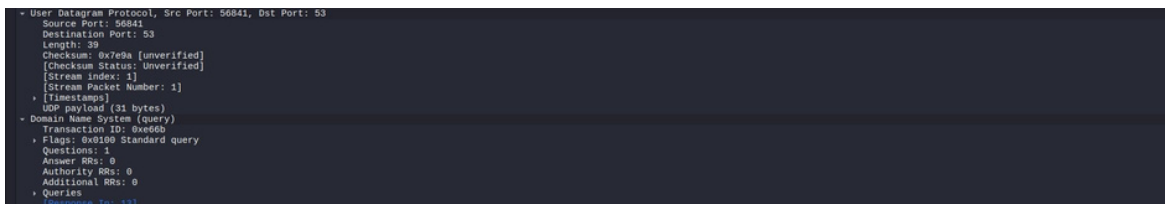


Fig. 4 – Dettaglio User Datagram Protocol e intestazione Domain Name System (query).

Porte di origine e destinazione	Origine: 56841 (porta effimera/dinamica) — Destinazione: 53
Numero di porta DNS predefinito	53

Per verificare la corrispondenza tra gli indirizzi osservati in Wireshark e quelli reali della mia macchina, ho eseguito il comando `ip address` in un terminale:

```
(kali@kali)~$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 5e:fd:b2:0a:85:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.60.3/24 brd 192.168.60.255 scope global dynamic noprefixroute eth0
        valid_lft 3066sec preferred_lft 2616sec
    inet6 fd8:6549:5802:eb41:48f0:9a96:e647:7b95/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 2591963sec preferred_lft 604763sec
    inet6 fe80::5b5f:4f04:d276:2463/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether f6:f8:db:5b:3b:ce brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth1
        valid_lft forever preferred_lft forever
```

Fig. 5 – Output di "ip address": conferma di MAC e IP dell'interfaccia eth0.

L'output conferma che l'interfaccia eth0 ha MAC 5e:fd:b2:0a:85:05 e IP 192.168.60.3/24, esattamente gli stessi valori osservati come origine nel pacchetto di query DNS. Questo dimostra che Wireshark stava effettivamente catturando il traffico generato dalla mia macchina e non da altri host della rete.

Infine ho espanso Domain Name System (query), Flags e Queries per osservare il comportamento richiesto al resolver e il tipo di record interrogato.

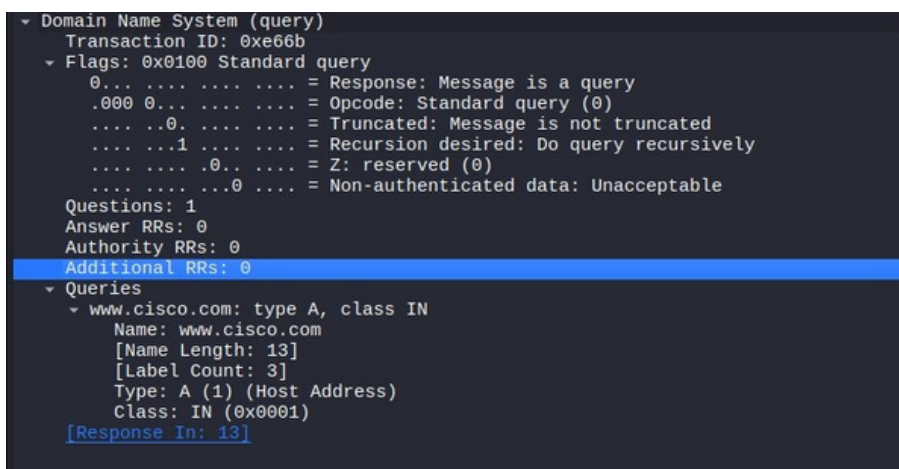


Fig. 6 – Flags e Queries del pacchetto DNS di richiesta.

Il flag Recursion desired risulta impostato a 1 ("Do query recursively"): il client richiede al resolver di eseguire la risoluzione in modo ricorsivo, ossia di interrogare a sua volta gli altri server della gerarchia DNS (root, TLD, autoritativi) se necessario, restituendo direttamente la risposta finale invece di un semplice riferimento al passo successivo. Il campo Queries indica inoltre che si tratta di una richiesta per il nome `www.cisco.com`, di tipo A (indirizzo IPv4), classe IN (Internet).

4. Parte 3 – Esplorare il Traffico delle Risposte DNS

Ho selezionato il pacchetto di risposta corrispondente (Frame 13, "Standard query response 0xe66b A www.cisco.com") ed espanso le medesime sezioni viste per la query.

```

▶ Frame 13: Packet, 255 bytes on wire (2040 bits), 255 bytes captured (2040 bits) on interface 0
  ▶ Ethernet II, Src: 86:2f:57:14:c9:64 (86:2f:57:14:c9:64), Dst: 5e:fd:b2:0a:85:05
    ▶ Destination: 5e:fd:b2:0a:85:05 (5e:fd:b2:0a:85:05)
    ▶ Source: 86:2f:57:14:c9:64 (86:2f:57:14:c9:64)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  ▶ Internet Protocol Version 4, Src: 192.168.60.1, Dst: 192.168.60.3
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      Total Length: 241
      Identification: 0x0013 (19)
    ▶ 000. .... = Flags: 0x0
      ...0 0000 0000 0000 = Fragment Offset: 0
      Time to Live: 64
      Protocol: UDP (17)
      Header Checksum: 0x8094 [validation disabled]
      [Header checksum status: Unverified]
      Source Address: 192.168.60.1
      Destination Address: 192.168.60.3
      [Stream index: 2]
  ▶ User Datagram Protocol, Src Port: 53, Dst Port: 56841
  ▶ Domain Name System (response)
    Transaction ID: 0xe66b
    ▶ Flags: 0x8180 Standard query response, No error
      1... .. = Response: Message is a response
      .000 0... .. = Opcode: Standard query (0)
      .... 0... .. = Authoritative: Server is not an authority for domain
      .... ..0... .. = Truncated: Message is not truncated
      .... ..1... .. = Recursion desired: Do query recursively
      .... ..1... .. = Recursion available: Server can do recursive queries
      .... ..0... .. = Z: reserved (0)
      .... ..0... .. = Answer authenticated: Answer/authority portion was not authenticated
      .... ..0... .. = Non-authenticated data: Unacceptable
      .... ..0000 = Reply code: No error (0)
    Questions: 1
    Answer RRs: 5
    Authority RRs: 0
    Additional RRs: 0
  ▶ Queries
    ▶ www.cisco.com: type A, class IN
      Name: www.cisco.com
      [Name Length: 13]
      [Label Count: 3]
      Type: A (1) (Host Address)
      Class: IN (0x0001)
  ▶ Answers
    [Request In: 12]
    [Time: 51.406346 milliseconds]
```

Fig. 7 – Dettaglio completo del Frame 13: intestazioni e Flags della risposta DNS.

Indirizzi MAC di origine e destinazione	Origine: 86:2f:57:14:c9:64 — Destinazione: 5e:fd:b2:0a:85:05
Indirizzi IP di origine e destinazione	Origine: 192.168.60.1 — Destinazione: 192.168.60.3
Porte di origine e destinazione	Origine: 53 — Destinazione: 56841
Confronto con il pacchetto di query	Tutti i valori risultano esattamente invertiti rispetto alla query: il server DNS (192.168.60.1) risponde dalla propria porta standard 53 verso la stessa porta effimera 56841, utilizzando anche la stessa porta di risposta diretta alla query osservata in precedenza.

Il server DNS può fare query ricorsive?

Sì. Nei Flags della risposta è impostato il bit "Recursion available: Server can do recursive queries", che indica la capacità del resolver di eseguire risoluzioni ricorsive per conto dei client.

Espandendo la sezione Answers del pacchetto di risposta, ho osservato la catena completa di record restituiti dal resolver:

```
Answers
  www.cisco.com: type CNAME, class IN, cname www.cisco.com.akadns.net
  www.cisco.com.akadns.net: type CNAME, class IN, cname wwwds.cisco.com.edgekey.net
  wwwds.cisco.com.edgekey.net: type CNAME, class IN, cname wwwds.cisco.com.edgekey.net.globalredir.akadns.net
  wwwds.cisco.com.edgekey.net.globalredir.akadns.net: type CNAME, class IN, cname e2867.dsca.akamaiedge.net
  e2867.dsca.akamaiedge.net: type A, class IN, addr 184.25.52.119
[Request In: 12]
[Time: 51.406346 milliseconds]
```

Fig. 8 – Sezione Answers: catena di record CNAME e record A finale.

La risposta contiene 5 Answer RR (Answer RRs: 5): quattro record CNAME concatenati e un record A finale, secondo lo schema seguente:

www.cisco.com	CNAME → www.cisco.com.akadns.net
www.cisco.com.akadns.net	CNAME → wwwds.cisco.com.edgekey.net
wwwds.cisco.com.edgekey.net	CNAME → wwwds.cisco.com.edgekey.net.globalredir.akadns.net
wwwds.cisco.com.edgekey.net.globalredir.akadns.net	CNAME → e2867.dsca.akamaiedge.net
e2867.dsca.akamaiedge.net	A → 184.25.52.119

Questa catena di reindirizzamenti è tipica dei siti che si appoggiano a una rete di distribuzione di contenuti (CDN): i domini intermedi akadns.net, edgekey.net e akamaiedge.net sono infrastrutture Akamai, alla quale Cisco delega la distribuzione dei contenuti del proprio sito web.

Come si confrontano i risultati con quelli di nslookup?

Confrontando la catena osservata in Wireshark con l'output di nslookup mostrato in Fig. 1, i due risultati coincidono perfettamente: stessa sequenza di nomi canonici (canonical name) e medesimo indirizzo IPv4 finale 184.25.52.119. nslookup mostra inoltre direttamente anche gli indirizzi IPv6 associati (2a02:26f0:2d:a92::b33 e 2a02:26f0:2d:a81::b33), corrispondenti ai record AAAA osservati separatamente nella cattura (frame 14 e 15, non analizzati nel dettaglio in questa relazione). La differenza principale tra i due strumenti è il livello di dettaglio: nslookup restituisce un riepilogo testuale leggibile della risoluzione, mentre Wireshark mostra la struttura completa del singolo pacchetto DNS, inclusi i flag di protocollo, i numeri di porta e gli indirizzi di livello 2 e 3 coinvolti.

5. Domande di Riflessione

1. Dai risultati di Wireshark, cos'altro puoi imparare sulla rete quando rimuovi il filtro?

Rimuovendo il filtro `udp.port == 53` torna visibile l'intera cattura, non solo il traffico DNS. Osservando la cattura completa è possibile notare la presenza di altri protocolli generati dalla macchina in background, come NTP per la sincronizzazione dell'orario, messaggi ICMPv6 Router Advertisement, traffico ARP per la risoluzione degli indirizzi MAC sulla rete locale ed eventuali segmenti TCP con ritrasmissioni, indice di possibili problemi di rete o di raggiungibilità verso alcuni host. In generale, senza filtro è possibile farsi un'idea complessiva del comportamento di rete di un host: quali altri servizi contatta, con quale frequenza, quali protocolli utilizza e, dal punto di vista della sicurezza, se è presente traffico trasmesso in chiaro (come HTTP, FTP o Telnet) che esporrebbe dati sensibili a chiunque fosse in grado di catturare i medesimi pacchetti.

2. Come può un attaccante usare Wireshark per compromettere la sicurezza della tua rete?

Wireshark è uno strumento puramente passivo: cattura soltanto il traffico che riesce a raggiungere l'interfaccia di rete su cui è in esecuzione. Il rischio per la sicurezza nasce quindi quando un attaccante riesce a posizionarsi in un punto della rete da cui può intercettare traffico non destinato a lui. Su una rete switched, questo avviene tipicamente tramite un attacco di ARP spoofing/poisoning, con cui l'attaccante si spaccia per il gateway nei confronti della vittima (e viceversa), dirottando così il traffico della vittima attraverso la propria macchina prima di inoltrarlo a destinazione: una tecnica di Man-in-the-Middle che rende possibile l'analisi con Wireshark di pacchetti che normalmente non sarebbero visibili. Una volta ottenuta la cattura, un attaccante può estrarre credenziali o dati sensibili trasmessi su protocolli non cifrati, oppure compiere attività di reconnaissance osservando semplicemente le query DNS della vittima, come fatto in questo laboratorio: i nomi di dominio interrogati rivelano quali servizi e siti vengono visitati e, in un contesto aziendale, possono far emergere nomi di server interni o sottodomini utili per pianificare attacchi successivi più mirati. La principale mitigazione contro questo tipo di rischio è l'adozione di cifratura end-to-end (HTTPS, DNS over TLS/HTTPS, VPN) e il monitoraggio della rete per individuare comportamenti anomali come l'ARP spoofing.

6. Conclusioni

Questo laboratorio mi ha permesso di analizzare nel dettaglio una risoluzione DNS completa, dalla query iniziale generata da nslookup fino alla risposta del resolver, osservando indirizzi MAC e IP, porte, flag di controllo e la catena di record CNAME tipica dei servizi distribuiti tramite CDN. Comprendere la struttura normale del traffico DNS è un prerequisito fondamentale in ambito SOC/CyberOps, poiché questo protocollo è frequentemente sfruttato sia per finalità offensive (DNS tunneling, cache poisoning, comunicazioni di Command and Control mascherate da query DNS) sia monitorato in ambito defensive per individuare comportamenti anomali, come query verso domini generati algebricamente o volumi di traffico DNS anomali.